

Document

CONTENTS

Architecture 1: Connectionist Temporal Classification (CTC)

Architecture 2: Attention-Based Encoder-Decoder (AED / LAS)

Architecture 3: RNN-Transducer (RNN-T)

Whisper Architecture: Why It's Non-Streaming

Code-Switching Within Single Utterances

Architecture Comparison Summary

Architecture 1: Connectionist Temporal Classification (CTC)

Mechanism

CTC predicts a token (or blank) for every audio frame independently, then collapses repeats and removes blanks to produce the final transcript. The loss function sums over all valid alignments between audio frames and output tokens using dynamic programming.

[arunbaby](#)

Key assumptions:

- **Conditional independence:** Each frame's prediction depends only on audio at that frame, not on previous tokens [arunbaby](#)
- **Monotonic alignment:** Output tokens are emitted left-to-right without reordering [arunbaby](#)

Streaming Capability

Native streaming: CTC is inherently streaming-friendly because it emits per-frame predictions without waiting for future context. Greedy decoding (argmax per frame) produces text as audio arrives. [arunbaby](#)

Latency characteristics:

- **Frame-level latency:** 10–20 ms per frame (typical frame shift) [heardright](#)
- **No lookahead required:** Can operate with zero right-context, though accuracy improves with 100–300 ms lookahead [arunbaby](#)

Typical Accuracy

CTC models achieve **6–10% WER** on clean speech (e.g., LibriSpeech), but struggle with:

- Ambiguous homophones ("read" vs. "red") due to weak language modeling [arunbaby](#)
- Rare words and proper nouns without external language model biasing [arunbaby](#)

Accuracy boosters:

- **Shallow fusion:** Combine CTC scores with external n-gram or neural LM at decode time ($\lambda = 0.1\text{--}0.5$) [arunbaby](#)
- **Contextual biasing:** Boost expected phrases (contact names, domain terms) in beam search [arunbaby](#)

PROPERTY	CTC
Streaming output	Yes (native) heardright
Language model	Weak (conditional independence) arunbaby
Latency	10–20 ms per frame heardright
Typical WER (LibriSpeech)	6–10% linkedin
On-device friendly	Yes (simple greedy decoding) heardright

Architecture 2: Attention-Based Encoder-Decoder (AED / LAS)

Mechanism

Listen, Attend, and Spell (LAS) models use a bidirectional encoder (LSTM, Conformer, or Transformer) to process the full audio sequence, then an autoregressive decoder attends to encoder states to generate text token-by-token. [arunbaby](#)

Key properties:

- **Global context:** Decoder attends to all encoder states, capturing long-range dependencies [arunbaby](#)
- **Autoregressive:** Each token conditions on previous tokens via decoder self-attention [arunbaby](#)

Streaming Capability

Not natively streaming: The decoder requires the full encoder output before generating the first token. Bidirectional attention means the encoder must see the entire utterance. [arunbaby](#)

Workarounds for pseudo-streaming:

- **Chunked attention:** Process audio in overlapping 1–2 second chunks with limited right-context (e.g., 500 ms lookahead) [arunbaby](#)
- **Sliding window:** Re-run encoder on rolling 30-second windows, emitting partials from each window [made-in-jurgistan.github](#)
- **Triggered attention:** Use a separate VAD or CTC model to trigger encoder-decoder inference on speech segments [arunbaby](#)

Latency Characteristics

Batch-oriented: TTFT (Time to First Token) scales linearly with utterance length because the encoder must process all frames before decoder starts. [made-in-jurgistan.github](#)

COMPONENT	LATENCY
Encoder (30s audio)	100–300 ms on GPU made-in-jurgistan.github
Decoder first token	+50–150 ms made-in-jurgistan.github
Total TTFT (30s utterance)	150–450 ms made-in-jurgistan.github

Typical Accuracy

AED models achieve **state-of-the-art accuracy** on offline transcription:

- **Whisper Large v3:** 7.44% WER (Open ASR Leaderboard average) [made-in-jurgistan.github](#)
- **Canary Qwen 2.5B:** 5.63% WER (current SOTA) [made-in-jurgistan.github](#)

Strengths:

- Strong language modeling via cross-attention [arunbaby](#)
- Multilingual support (Whisper: 99 languages) [made-in-jurgistan.github](#)
- Prompt-driven language control (e.g., <|en|> , <|translate|>) [heardright](#)

PROPERTY	ENCODER-DECODER (WHISPER)
Streaming output	No (batch) heardright
Language control via prompt	Yes heardright
Translation to English	Yes heardright
Hallucination risk on silence	Higher heardright
Typical WER (LibriSpeech)	3–6% arunbaby

Architecture 3: RNN-Transducer (RNN-T)

Mechanism

RNN-T combines three components:

1. **Encoder (Transcription Network):** Processes audio frames (similar to CTC encoder) [arunbaby](#)
2. **Prediction Network:** Autoregressive LM that conditions on previously emitted tokens [arunbaby](#)
3. **Joint Network:** Combines encoder and prediction outputs to emit tokens [arunbaby](#)

Key innovation: The prediction network acts as an **internal language model**, removing CTC's conditional independence assumption while preserving streaming capability.

[arunbaby](#)

Streaming Capability

Native streaming: RNN-T emits tokens left-to-right as audio arrives, with bounded latency. The model can only move forward in time (monotonic), making it ideal for real-time applications. [arunbaby](#)

Decoding grid: Imagine a 2D grid where:

- X-axis: Time frames (T)
- Y-axis: Output tokens (U)
- At each step, model can emit token (move up) or blank (move right) [arunbaby](#)

Latency Characteristics

Frame-synchronous: Emits tokens every 10–40 ms with internal LM context. Latency is tunable via lookahead:

- **0 ms lookahead:** Lowest latency, but accuracy drops (can't distinguish "The" vs. "A" without future context) [arunbaby](#)
- **100–300 ms lookahead:** Sweet spot for production systems [arunbaby](#)

Production deployments:

- **Google Assistant, Apple Siri:** RNN-T with ~200 ms latency [valmera](#)
- **NVIDIA Parakeet TDT:** RTFx > 2000 on A100 (2,000× faster than real-time) [made-in-jurgistan.github](#)

Typical Accuracy

RNN-T models achieve **better accuracy than CTC** (due to internal LM) while maintaining streaming:

- **NVIDIA Parakeet TDT 1.1B:** 7.02% WER (Open ASR average) [made-in-jurgistan.github](#)
- **Google's production ASR:** ~5–7% WER on conversational speech [cheatgrid](#)

PROPERTY	RNN-T / TDT
Streaming output	Yes (native) heardright
Built-in language model	Yes (prediction network) arunbaby
Latency	100–300 ms (tunable) arunbaby
Typical WER (LibriSpeech)	4–7% arunbaby
On-device friendly	Yes (streaming, no full attention) heardright

Whisper Architecture: Why It's Non-Streaming

Encoder-Decoder with 30-Second Windows

Whisper (OpenAI, 2022) is a standard Transformer encoder-decoder trained on 680,000 hours of weakly-labeled web audio. Key design choices: [made-in-jurgistan.github](#)

1. **Fixed 30-second input:** Encoder always processes exactly 30 seconds of audio (padded with silence if shorter) [made-in-jurgistan.github](#)
2. **Bidirectional attention:** Encoder attends to all 30 seconds simultaneously; decoder attends to full encoder output [made-in-jurgistan.github](#)
3. **Autoregressive decoder:** Generates tokens one-by-one, conditioning on all previous tokens [made-in-jurgistan.github](#)

Why True Streaming Is Difficult

Fundamental bottleneck: The decoder cannot start until the encoder finishes processing all 30 seconds. This creates:

- **Linear TTFT growth:** A 2-second utterance still waits for 30 seconds of encoder compute (28 seconds wasted on padding) [made-in-jurgistan.github](#)
- **No incremental output:** Cannot emit partials while user speaks; must buffer entire utterance [made-in-jurgistan.github](#)
- **Hallucination on silence:** Decoder was trained on real audio and generates plausible text for random noise/silence [made-in-jurgistan.github](#)

Techniques to Approximate Streaming

TECHNIQUE	DESCRIPTION	TRADE-OFFS
Chunked inference	Split audio into overlapping 5–10 second chunks; run Whisper on each chunk independently made-in-jurgistan.github	Loses long-range context; words at chunk boundaries may be split or dropped
Sliding window	Process 30-second windows with 5–10 second stride; merge overlapping transcripts made-in-jurgistan.github	Redundant compute (re-processes same audio); latency still 30s per window
Whisper-streaming	Use CTC or RNN-T front-end to detect speech segments; run Whisper only on detected segments yobitel	Adds complexity; still batch-oriented within each segment
Distil-Whisper	Smaller decoder (4 layers vs. 32) reduces TTFT but preserves encoder accuracy made-in-jurgistan.github	Still non-streaming; just faster batch inference

Example: `whisper.cpp` implements chunked streaming by:

1. Buffering audio into 30-second segments
2. Overlapping adjacent chunks by 1–2 seconds to avoid boundary artifacts
3. Emitting partials after each chunk completes [made-in-jurgistan.github](#)

Code-Switching Within Single Utterances

What Is Code-Switching?

Code-switching (CSW) is the alternation of languages within a single utterance (e.g., "I need to book a flight *بكره*" mixing English and Arabic). This is common in multilingual societies (India, Middle East, Southeast Asia) but remains a fundamental challenge for ASR. [arxiv](#)

Why It's Architecturally Hard

1. **Language ID ambiguity:** Monolingual models assume single language per utterance; CSW violates this assumption [arxiv](#)

- 2. **Vocabulary explosion:** Bilingual models need combined vocabulary (e.g., English + Hindi = 100K+ tokens), increasing model size and training data requirements [arxiv](#)
- 3. **Acoustic-linguistic mismatch:** Phonemes from language A may be pronounced with language B's prosody, confusing acoustic models [lrec.elra](#)
- 4. **LM context switching:** Language models trained on monolingual text struggle to predict code-switched sequences ("I'm going to *المكتب*" → next word could be English or Arabic) [arxiv](#)

Modern Approaches

APPROACH	DESCRIPTION	LIMITATIONS
Multilingual training	Train on mixed-language corpora (e.g., Google's Universal Speech Model) arxiv	Requires massive CSW datasets; most public datasets are monolingual
Language-adaptive layers	Add language-specific adapter modules to shared encoder arxiv	Increases model complexity; requires language tags at inference
CSW-aware LMs	Train external LMs on code-switched text; use shallow fusion arxiv	Limited CSW text corpora; most web text is monolingual
Token-level language ID	Predict language ID per token alongside transcription lrec.elra	Adds output dimension; requires token-level language annotations

State-of-the-art: Most production ASR systems (Whisper, Google Cloud STT) handle CSW poorly—WER degrades 20–50% on mixed-language utterances vs. monolingual. [arxiv](#)

Architecture Comparison Summary

PROPERTY	CTC	RNN-T / TDT	ENCODER-DECODER (WHISPER)
Streaming output	Yes (native) heardright	Yes (native) heardright	No (batch) heardright
Language control via prompt	No heardright	No heardright	Yes heardright

PROPERTY	CTC	RNN-T / TDT	ENCODER-DECODER (WHISPER)
Translation to English	No heardright	No heardright	Yes heardright
Built-in language model	No (needs external LM) arunbaby	Yes (prediction network) arunbaby	Yes (cross-attention) arunbaby
Hallucination risk on silence	Low heardright	Low heardright	Higher heardright
Typical WER (LibriSpeech)	6–10% linkedin	4–7% arunbaby	3–6% arunbaby
Latency	10–20 ms per frame heardright	100–300 ms (tunable) arunbaby	150–450 ms (30s utterance) made-in-jurgistan.github
On-device friendly	Yes heardright	Yes heardright	⚠ High-end only made-in-jurgistan.github
Code-switching support	Poor arxiv	Poor arxiv	Moderate (multilingual training) arxiv

Key papers:

- **CTC:** Graves et al. (2006), "Connectionist Temporal Classification" [arunbaby](#)
- **RNN-T:** Graves (2012), "Sequence Transduction with Recurrent Neural Networks" [arunbaby](#)
- **LAS:** Chan et al. (2015), "Listen, Attend and Spell" [arunbaby](#)
- **Whisper:** Radford et al. (2022), "Robust Speech Recognition via Large-Scale Weak Supervision" [made-in-jurgistan.github](#)
- **Code-switching survey:** LingO-IITGN (2025), "Beyond Monolingual Assumptions: A Survey of Code-Switched NLP" [arxiv](#)

Production systems in 2026 increasingly use **hybrid architectures**: RNN-T for streaming partials + encoder-decoder for final high-accuracy pass, or multilingual Whisper with language-adaptive layers for CSW handling. [arxiv](#)

